

To implement inner classes and anonymous classes for improving encapsulation and event handling in Java.

Theory :

An inner class is a class defined inside another class and has access to all members of the enclosing class, including private members. Inner classes help improve code organization and logical grouping of related classes. Anonymous classes are unnamed inner classes that are instantiated only once and are mainly used for implementing interfaces or abstract classes. They simplify code by eliminating the need for separate class definitions. These concepts are widely used in Java GUI programming and event handling

Outcome :

- Understood the concept and implementation of inner classes and anonymous classes in Java.
- Learned how inner classes improve encapsulation and code organization.
- Implemented anonymous classes for one-time actions and event handling.
- Developed practical applications using these concepts.

Conclusion :

The Java application was successfully developed using inner classes and anonymous classes. Inner classes were used to improve encapsulation by organizing related data and methods within the enclosing class, while anonymous classes were used to perform one-time actions without creating separate class definitions. The Vehicle and Food Delivery applications demonstrated the practical use of these concepts, improving code organization, reusability, and event handling in Java.

1. Create a Vehicle program where an inner class displays vehicle details

and anonymous class performs an action.

```
class Vehicle {
    String name = "Car";
    int speed = 120;

    class Details {
        void display() {
            System.out.println("Vehicle Name: " + name);
            System.out.println("Speed: " + speed + " km/h");
        }
    }
}

interface Action {
    void perform();
}

public static void main(String[] args) {
    Vehicle v = new Vehicle();

    Vehicle.Details d = v.new Details();
    d.display();

    Action a = new Action() {
        public void perform() {
            System.out.println("Vehicle is Running");
        }
    };

    a.perform();
}
```

```
}  
}
```

OUTPUT

Vehicle Name: Car
Speed: 120 km/h
Vehicle is Running

2. Develop a Food Delivery Application where an inner class handles order details and anonymous classes handle delivery status updates.

```
class FoodDelivery {  
    String orderId = "FD101";  
    String foodItem = "Pizza";  
  
    class OrderDetails {  
        void display() {  
            System.out.println("Order ID: " + orderId);  
            System.out.println("Food Item: " + foodItem);  
        }  
    }  
}  
  
interface DeliveryStatus {  
    void status();  
}  
  
public static void main(String[] args) {  
    FoodDelivery f = new FoodDelivery();
```

```
FoodDelivery.OrderDetails o = f.new OrderDetails();  
o.display();
```

```
DeliveryStatus d = new DeliveryStatus() {  
    public void status() {  
        System.out.println("Order Delivered Successfully");  
    }  
};
```

```
    d.status();  
}  
}
```

OUTPUT

Order ID: FD101

Food Item: Pizza

Order Delivered Successfully